

SMART CAMPUS & PLACEMENT PLATFORM

Documentation Update — V1.2: Foundation + API Architecture

1. Purpose

This document records the implementation progress after the original project blueprint. It is a historical progress snapshot: what was built, what was learned, and how the architecture changed.

The original documentation was Version 1.1 of the project blueprint and described the intended product, target architecture, domain model, technology stack, roadmap, and learning method. This update records the work completed after that baseline. The attached original blueprint identifies DTOs and mappers under the professional-architecture phase and describes the intended layered request flow. (Original blueprint: pages 3–5.)

2. Important versioning clarification

The documentation version and the project version are different. This is **Documentation V1.2**. The actual application is still in its foundational stage. We will call the next major functional/domain expansion **Project V2** rather than pretending that every architecture improvement is a new product version.

3. Current implemented stack

Area	Current implementation
Language	Java
Build	Maven
Framework	Spring Boot 4.1.1
Web	Spring MVC / REST
Persistence	JPA + Hibernate
Data access	Spring Data JPA
Database	PostgreSQL
Validation	Bean Validation
API testing	Postman
Version control	Git + GitHub

4. Completed foundation

- Created the Spring Boot application with Maven and Spring Initializr.
- Configured Spring Web/MVC, Spring Data JPA, PostgreSQL, and validation support.
- Connected the application to the smart_campus_platform PostgreSQL database.
- Created the Student JPA entity with a database-generated primary key.
- Created StudentRepository using JpaRepository.
- Built StudentController → StudentService → StudentRepository layered CRUD.
- Tested POST, GET all, GET by ID, PUT, and DELETE using Postman.

5. Validation and exception handling

- Moved request validation to the API boundary using StudentRequest.
- Used @NotBlank for required text fields and @Email + @NotBlank for email.
- Used @Min(2020) and @Max(2035) for graduation year.

- Created `StudentNotFoundException` for missing students.
- Created `GlobalExceptionHandler` with `@RestControllerAdvice`.
- Validation failures return a structured 400 response with field-level errors.
- Missing students return a predictable 404 response.

6. DTO architecture

The major architectural change was separating API models from persistence entities. `StudentRequest` represents incoming client data. `Student` represents persisted JPA data. `StudentResponse` represents outgoing API data.

Model	Responsibility	ID
<code>StudentRequest</code>	Incoming API contract	No ID
<code>Student</code>	JPA persistence entity	Generated by DB
<code>StudentResponse</code>	Outgoing API contract	Includes ID

This reduces coupling: database structure can evolve without forcing the public API contract to evolve in exactly the same way, and API-specific fields do not have to become persistence fields.

7. Mapper + Spring DI

- Created `StudentMapper` with `StudentRequest` → `Student` conversion.
- Added `Student` → `StudentResponse` conversion.
- Made `StudentMapper` a Spring bean using `@Component`.
- Injected `StudentMapper` into `StudentService` using constructor injection.
- Removed manual `new StudentMapper()` construction.

8. Current API data flow

Request: HTTP JSON → `StudentRequest` → `StudentMapper` → `Student Entity` → `Repository` → PostgreSQL.

Response: PostgreSQL → `Student Entity` → `StudentMapper` → `StudentResponse` → HTTP JSON.

List response: `List<Student>` → `stream()` → `map(studentMapper::toResponse)` → `toList()` → `List<StudentResponse>`.

9. Design decisions captured

- The generated `Student` ID is readable through `getId()`, but there is intentionally no `setId()` on the entity.
- `StudentResponse` can receive the generated ID because the application constructs the response DTO.
- JPA entities and request DTOs retain no-argument constructors for framework object construction.
- Business/application coordination remains in the service instead of the controller.

10. Status at this checkpoint

All five `Student` CRUD operations have been tested successfully after the DTO changes. The `Student` API now has a consistent request/response boundary rather than exposing the JPA entity directly.

Next major step: expand the isolated `Student` feature into the connected campus-placement domain. That is the point at which **Project V2** begins.

Historical note: this document should not be overwritten by future versions. Keep it as a record of how the project evolved.